Bispectrum Library: From Finite Groups to the Disk

Implementing Mataigne et al. (NeurIPS 2024) & Myers & Miolane (arXiv 2025)

Johan Mathe

March 9, 2026

Goal

Reimplement the selective G-bispectrum from Mataigne et al. (NeurIPS 2024) in a standalone, clean PyTorch library.

Three group/domain families implemented:

- **CnonCn** — cyclic groups C_n acting on $\mathbb{Z}/n\mathbb{Z}$ *Mataigne et al.*
- **DnonDn** — dihedral groups D_n acting on D_n *Mataigne et al.*
- **SO2onD2** — $\mathrm{SO}(2)$ acting on the unit disk $\mathbb{D}^2$ *Myers & Miolane*

All three include `forward()` (bispectrum) and `invert()` (signal recovery).

Plus **SO3onS2** ($\mathrm{SO}(3)$ on S^2) — forward only, no inversion.

Improvements over g-invariance repo

Aspect	**g-invariance**	**bispectrum (new)**
Dependencies	escnn, einops, scipy	Pure PyTorch
CG computation	`scipy.linalg.schur`	`torch.linalg.eig`
Group size	$n \geq 8$ enforced	All $n > 2$
n_3 for odd n	Bug: gives 0 for $n = 3$	Fixed
Inversion	Not implemented	Algorithms 2 & 4
Interface	escnn `GeometricTensor`	Raw `torch.Tensor`
Normalization	Baked in	None (raw coefficients)
Group DFT	$O(n^2)$ explicit	$O(n \log n)$ via FFT
Tests	**None**	**180 tests**

Bugs found in g-invariance (1/2)

Bug 1: n_3 calculation breaks for small odd n

g-invariance code: `n3 = n2 - 1` where $n_2 = \lfloor (n-1)/2 \rfloor$.

For $n = 3$: $n_2 = 1$, so $n_3 = 0$ — meaning *zero* β_{ρ_1,ρ_k} coefficients are computed.

Impact: β_{ρ_1,ρ_1} is needed to recover $\mathcal{F}(\rho_{01})$ (Appendix E of the paper proves $\rho_{01} \in \rho_1 \otimes \rho_1$). Without it, the selective bispectrum for D_3 is *incomplete* — the 1D irrep ρ_{01} cannot be recovered during inversion.

How discovered: Writing parametrized tests over $n \in \{3, 4, 5, 7, 8\}$. D_3 inversion failed because `output_size` was only $1 + 4 = 5$ (missing the $16 \times n_3$ block entirely). The g-invariance repo never tested $n < 8$ due to bug 2.

Fix: `n3 = n2 if n % 2 == 0 else max(n2 - 1, 1)`.

Bugs found in g-invariance (2/2)

Bug 2: Artificial $n \geq 8$ restriction

`pooling.py` line 390: `if n < 8: raise ValueError`.

This came from escnn integration constraints, not from the math. D_n bispectrum is well-defined for all $n > 2$. Dropping this restriction immediately exposed Bug 1.

Bug 3: Paper typo in Algorithm 4 (next slide).

Note: The g-invariance repo had **zero unit tests**, so these bugs were never caught. Bug 1 is silent — it doesn't crash, it just computes an incomplete bispectrum.

Paper typo in Algorithm 4 (bispectrum inversion on D_n)

Algorithm 4, line 6 computes the block-diagonal Fourier coefficients via:

$$\bigoplus_{\rho \in \rho_1 \otimes \rho_{k-1}} \mathcal{F}(\Theta)_\rho = \left(C^\dagger_{\rho_1,\rho_{k-1}} \left[\mathcal{F}_{\rho_1} \otimes \mathcal{F}_{\rho_{k-1}} \right]^{-1} \beta_{\rho_1,\rho_2} \, C_{\rho_1,\rho_{k-1}} \right)^\dagger$$

The subscript on β is printed as ρ_2 (fixed), but should be ρ_{k-1} (varying with the loop index k). The CG subscripts $C_{\rho_1,\rho_{k-1}}$ are correct.

How discovered: When implementing inversion for $n = 8$ (which has $\lfloor \frac{7}{2} \rfloor = 3$ 2D irreps), the loop needs β_{ρ_1,ρ_1} then β_{ρ_1,ρ_2} to recover $\mathcal{F}(\rho_2)$ then $\mathcal{F}(\rho_3)$. Using a fixed β_{ρ_1,ρ_2} for both iterations gives wrong Fourier coefficients — the Frobenius norm roundtrip test fails.

The g-invariance code indexes β by `k-1` in the loop, so it was already correct despite the paper typo.

Technical challenges: CG matrices without scipy

Goal: find orthogonal C such that $(\rho_i \otimes \rho_j)(g) = C\left[\bigoplus_\rho \rho(g)\right] C^\top$ for all $g \in D_n$.

The g-invariance repo used `scipy.linalg.schur`. We need pure PyTorch.

Step 1: Eigendecompose $Q_a = \mathrm{kron}(\rho_i(a), \rho_j(a))$ via `torch.linalg.eig`. This 4×4 orthogonal matrix has eigenvalues $e^{\pm i\theta}$ (conjugate pairs $\to$ 2D irreps) or ± 1 (real $\to$ 1D irreps).

Step 2 — 2D blocks: For each conjugate pair $e^{\pm i\theta}$, take $u_1 = \mathrm{Re}(v)$, $u_2 = \mathrm{Im}(v)$. In this basis the block is $R(-\theta)$, but our irrep convention is $\rho_k(a) = R(+2\pi k/n)$. Fix: negate u_2 when $\theta > 0$ so the block becomes $R(+|\theta|)$.

Step 3 — 1D blocks: When two 1D irreps share the same eigenvalue under a (e.g. ρ_0 and ρ_{01} both have $+1$), the eigenspace is 2D and `eig` won't split them. We restrict $Q_x = \mathrm{kron}(\rho_i(x), \rho_j(x))$ to the degenerate subspace and diagonalize with `eigh`; its eigenvalues ± 1 separate the two 1D irreps.

Technical challenges: bispectrum formula

Non-commutative bispectrum (Theorem 2.3):

$$\beta_{\rho_1,\rho_2} = \left[\mathcal{F}_{\rho_1} \otimes \mathcal{F}_{\rho_2}\right] C \left[\bigoplus_{\rho} \mathcal{F}_{\rho}^{\top}\right] C^{\top}$$

Getting the transpose/adjoint conventions right for real-valued irreps required several iterations. Initial implementation broke D_n invariance due to incorrect matrix multiplication order.

Verified by testing invariance under all n rotations and reflections for $n \in \{3, 4, 5, 7, 8, 16\}$.

Technical challenges: $O(2)$ indeterminacy in inversion

Algorithm 4 recovers $\mathcal{F}(\rho_1)$ from:

$$\frac{\beta_{\rho_0,\rho_1}}{\mathcal{F}(\rho_0)} = \mathcal{F}(\rho_1)^\top \mathcal{F}(\rho_1)$$

This determines $\mathcal{F}(\rho_1)$ only up to $Q \in O(2)$: any $Q \cdot \mathcal{F}(\rho_1)$ also satisfies the equation.

Consequence: exact bispectrum roundtrip $\beta \to$ invert $\to$ forward $\to \beta' = \beta$ cannot hold for all coefficients. This is a *theoretical* limitation, not a bug.

What we guarantee:

- β_{ρ_0,ρ_0} and β_{ρ_0,ρ_1} roundtrip exactly (they are $O(2)$-invariant)
- Frobenius norms $\|\mathcal{F}(\rho_k)\|_F$ match for all k

From group DFT to standard FFT: the derivation

The D_n group DFT for 2D irrep ρ_k sums over rotation and reflection elements:

$$\hat{f}(\rho_k) = \sum_{l=0}^{n-1} f_{\text{rot}}(l)\, R\left(\tfrac{2\pi kl}{n}\right)^\top + \sum_{l=0}^{n-1} f_{\text{ref}}(l)\, \left[R\left(\tfrac{2\pi kl}{n}\right) \operatorname{diag}(1,-1)\right]^\top$$

Expand the 2×2 entries, noting $R(\theta)^\top = \left[\begin{smallmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{smallmatrix}\right]$:

$$\hat{f}_{00}^{(k)} = \textstyle\sum_l f_{\text{rot}}(l) \cos\theta_{kl} + \sum_l f_{\text{ref}}(l) \cos\theta_{kl}$$

But $\texttt{fft}(f)[k] = \sum_l f(l)\, e^{-i2\pi kl/n}$, so $\operatorname{Re}(F_{\text{rot}}[k]) = \sum_l f_{\text{rot}}(l) \cos\theta_{kl}$.

Therefore all four matrix entries are Re/Im combinations of two standard FFTs:

$$\hat{f}^{(k)} = \begin{bmatrix} \operatorname{Re} F_r + \operatorname{Re} F_x & \operatorname{Im} F_r - \operatorname{Im} F_x \\ -\operatorname{Im} F_r - \operatorname{Im} F_x & \operatorname{Re} F_r - \operatorname{Re} F_x \end{bmatrix}$$

where $F_r = \texttt{fft}(f_{\text{rot}})$ and $F_x = \texttt{fft}(f_{\text{ref}})$. Two $O(n \log n)$ FFTs replace one $O(n^2)$ matrix–vector product.

FFT-based group DFT: 100–275× speedup

Replaced $O(n^2)$ explicit DFT (einsum against precomputed rotation matrices) with two calls to `torch.fft.fft` / `torch.fft.ifft`.

Key insight: each entry of the 2×2 Fourier coefficient $\hat{f}(\rho_k)$ is a linear combination of Re / Im of the standard FFT of the rotation and reflection halves of the signal.

Method	n	**Before**	**After**	**Speedup**
`_group_dft`	64	50 ms	0.23 ms	218×
`_group_dft`	256	76 ms	0.28 ms	275×
`_group_dft`	1024	89 ms	0.44 ms	205×
`_inverse_dft`	64	2.3 ms	0.27 ms	8.5×
`_inverse_dft`	256	10.5 ms	0.33 ms	32×
`_inverse_dft`	1024	71 ms	0.56 ms	127×

Also removes $O(n^2)$ memory from precomputed rotation buffers.

New: SO2onD2

Implementing the Selective Disk Bispectrum

Myers & Miolane, arXiv:2511.19706, 2025

SO2onD2: What and why

Setting: $f : \mathbb{D} \to \mathbb{R}$ (grayscale image on the unit disk). $\mathrm{SO}(2)$ acts by rotation: $(R_\phi f)(r, \theta) = f(r, \theta - \phi)$.

Disk Harmonic Transform (analog of Fourier/SHT):

$$f(r, \theta) = \sum_{n,k} a_{nk}\, \psi_{nk}(r, \theta), \qquad \psi_{nk} = c_{nk}\, J_n(\lambda_{nk} r)\, e^{in\theta}$$

Equivariance: $a_{n,k}^{R_\phi f} = e^{in\phi} \cdot a_{n,k}^{f}$

Selective disk bispectrum (Def. 4.2) — $O(m)$ coefficients:

$$b_{0,0,k} = a_{0,1}^2 \cdot a_{0,k}^*, \qquad b_{2,n,k} = a_{1,1} \cdot a_{n,1} \cdot a_{n+1,k}^*$$

Phases cancel: $e^{i\phi} \cdot e^{in\phi} \cdot e^{-i(n+1)\phi} = 1$. **Invertible** (Thm. 4.4).

SO2onD2: Architecture

Follows the same `nn.Module` contract as `CnonCn`/`DnonDn`:

- No trainable parameters (all buffers)
- `forward(f)` takes raw $L \times L$ images, handles DHT internally
- `invert(beta)` recovers image up to rotation
- `output_size`, `index_map` properties

Key design choices:

1. No new dependencies — Bessel functions from scratch in pure torch
2. No external data files — all roots & matrices computed at construction
3. `selective=False` raises `NotImplementedError` (as in `DnonDn`)
4. Mirrors `CnonCn` structure: $\mathrm{SO}(2)$ is commutative $\Rightarrow$ scalar triple products, no CG matrices

Challenge 1: Bessel functions without scipy

Need $J_n(x)$ and its positive zeros λ_{nk} where $J_n(\lambda_{nk}) = 0$.

$J_n(x)$: Forward recurrence from `torch.special.bessel_j0/j1`:

$$J_{k+1}(x) = \frac{2k}{x} J_k(x) - J_{k-1}(x)$$

Stable when $x \geq n$ (our regime: $x = \lambda_{nk} r$, $\lambda_{nk} > n$).

Root finding: **Interlacing property** $\lambda_{n-1,k} < \lambda_{n,k} < \lambda_{n-1,k+1}$ gives guaranteed brackets.

1. J_0 roots via McMahon expansion + Newton polish
2. For $n = 1, 2, \ldots$: bracket each J_n root between consecutive J_{n-1} roots
3. Newton–bisection hybrid within each bracket (guaranteed convergence)

Single-pass algorithm: `compute_all_bessel_roots(n_max, k_max)`.

Challenge 2: DHT on a discrete pixel grid

Problem: The natural complex basis $\{\psi_{nk}, \psi_{-n,k}\}$ is redundant for real signals ($\psi_{-n,k} = \overline{\psi_{n,k}}$). On a discrete grid, the synthesis matrix is *rank-deficient* (condition number $\sim 10^{20}$).

Solution: Reformulate using a **real basis**:

$$n = 0: \quad c_{0k} J_0(\lambda_{0k} r) (1 \text{ column per } (0, k))$$

$$n > 0: \quad 2\, c_{nk} J_n(\lambda_{nk} r) \cos(n\theta), \quad -2\, c_{nk} J_n(\lambda_{nk} r) \sin(n\theta) (2 \text{ columns})$$

Yields matrix $\Phi \in \mathbb{R}^{p_{\text{disk}} \times d_{\text{real}}}$, well-conditioned ($\kappa \sim 10^3$).

Analysis: $\mathbf{x} = \Phi^+ \mathbf{f}$ (pseudoinverse, `rcond=1e-10`)

Synthesis: $\mathbf{f} = \Phi\, \mathbf{x}$

Convert real $\mathbf{x}$ to complex $a_{n,k} = x_{\text{re}} + i\, x_{\text{im}}$ for the bispectrum.

Challenge 3: Bandlimit convention

Paper says $\lambda = \pi L/2$, but what determines m exactly?

Convention from Marshall et al. (2023, `fle-2d`):

$$n_e = \lfloor L^2 \pi / 4 \rfloor \approx \text{number of pixels inside the disk}$$

Sort all (n, k) by ascending λ_{nk}, keep the first n_e.

Edge case: if n_e splits a conjugate pair (n, k) and $(-n, k)$, drop the orphan.

Image size	$L = 8$	$L = 16$	$L = 28$	$L = 56$	$L = 112$
Paper (Table 1)	27	105	315	1,247	4,957
Our implementation	27	105	315	—	—

Exact match for all tested sizes. ($L \geq 56$ untested due to construction time.)

Inversion: bootstrap recovery (Thm. 4.4)

Same pattern as `CnonCn.invert()`, adapted to 2D indexing:

1. $a_{0,1}$ from $b_{0,0,1} = |a_{0,1}|^3 e^{i \arg(a_{0,1})}$ $\Rightarrow$ cube root
2. $a_{0,k}$ for $k \geq 2$: $a_{0,k} = \overline{b_{0,0,k}/a_{0,1}^2}$
3. $|a_{1,1}|$ from $b_{2,0,1} = a_{0,1}|a_{1,1}|^2$ (phase = rotation indeterminacy)
4. $a_{1,k}$ for $k \geq 2$: $a_{1,k} = \overline{b_{2,0,k}/(a_{1,1} \cdot a_{0,1})}$
5. Chain for $n \geq 1$: $a_{n+1,k} = \overline{b_{2,n,k}/(a_{1,1} \cdot a_{n,1})}$

At the coefficient level: exact (magnitudes match to 10^{-12}).

Full roundtrip ($f \to \beta \to \hat{f} \to f'$): limited by DHT discretization; >85% of DH coefficients recovered within 20% relative error.

Assumption: $a_{n,1} \neq 0$ for all n. Mild for noisy images; fails for purely radial signals.

SO2onD2: Usage

```
from bispectrum import SO2onD2

bsp = SO2onD2(L=28)           # 28x28 images
print(bsp.output_size)         # 315

f = torch.randn(4, 28, 28)    # batch of 4 images
beta = bsp(f)                  # (4, 315), complex128
f_rec = bsp.invert(beta)       # (4, 28, 28), float64

# Invariance check
cos_a, sin_a = math.cos(0.7), math.sin(0.7)
theta = torch.tensor([[[cos_a, sin_a, 0],
                        [-sin_a, cos_a, 0]]])
grid = F.affine_grid(theta, [1,1,28,28])
f_rot = F.grid_sample(f[:1].unsqueeze(1), grid)

beta_rot = bsp(f_rot.squeeze(1))
# beta and beta_rot are approximately equal
```

Numerical subtleties encountered

1. **Forward recurrence cancellation**: $J_n(x)$ near its own zeros loses $\sim$7 digits for $n \geq 2$. Our computed roots are zeros of the *numerically evaluated* J_n, not the true J_n. Everything is self-consistent; displacement $\sim 6 \times 10^{-5}$.
2. **dtype leak**: `torch.tensor(x)` with Python float infers `float32`, not `float64`. Silently halved precision of all root-finding.
3. **Phase precision**: `torch.exp(1j * n * phi)` creates `complex64` even when multiplied with `complex128` tensors. Must explicitly pass `dtype=complex128`.
4. **Pseudoinverse regularization**: 1–2 basis functions per grid are unresolvable (aliased at the pixel scale). Setting `rcond=1e-10` in `pinv` zeros their singular values instead of amplifying noise.

Test coverage: 180 tests

CnonCn (42 tests)

- Output shape/dtype (6 values of n)
- Cyclic shift invariance (selective & full, all shifts)
- Formula verification vs manual DFT
- Inversion: DFT magnitude roundtrip
- Inversion: full bispectrum roundtrip

DnonDn (67 tests)

- Rotation invariance (all shifts, 6 values of n)
- Reflection invariance (6 values of n)
- Combined rotation + reflection
- DFT roundtrip (forward + inverse = identity)
- Inversion: $\beta_{\rho_0,*}$ exact roundtrip
- Inversion: Fourier Frobenius norm recovery
- DFT performance benchmarks (n up to 1024)

SO2onD2 (45 tests)

- Bessel: J_n vs torch, root accuracy, interlacing
- Construction: coeff counts vs paper Table 1
- Forward: shape, dtype, analytical & spatial invariance
- DHT roundtrip: bandlimited & single harmonic
- Inversion: coefficient-level, bispectrum roundtrip

Other (32 tests)

- SO3onS2: construction, forward, rotation invariance
- Rotation utilities, import / public API surface

Weaknesses & areas for improvement

1. **No GPU testing** — all tests run on CPU only
2. D_n **inversion is partial** — $O(2)$ indeterminacy is fundamental; worth investigating canonical choices of Q
3. **Full bispectrum not implemented** — only `selective=True` works for `DnonDn` and `SO2onD2`
4. **SO2onD2 construction is slow** — Bessel root finding + pseudoinverse in pure Python; $L = 28$ takes $\sim$15s, $L = 64$ impractical
5. **Naive DHT** — $O(L^3)$ pseudoinverse instead of $O(L^2 \log L)$ fast DHT from Marshall et al. (2023)
6. **No G-CNN integration yet** — standalone module, not wired into an equivariant network pipeline

Next steps

- **SO2onD2: MRA correction** (Proposition 4.6) — bias term δ for noisy rotated observations
- **SO2onD2: fast DHT** — integrate $O(L^2 \log L)$ approximation from Marshall et al.
- **SO2onD2: performance** — cache Bessel roots, or move to C++/CUDA
- Wire into a G-CNN pipeline (replace escnn pooling layer)
- Reproduce MNIST/EMNIST/CIFAR-10 experiments from the papers
- Explore octahedral / full octahedral groups (Mataigne Appendix F)
- Investigate selective bispectrum for $\mathrm{SO}(3)$ on S^2 (open problem)
- GPU testing and benchmarking